=== UNIT 1: WEB ARCHITECTURE, HTML5 SEMANTICS & FORMS ===

>> 1.1 Client-Server Web Architecture and HTTP Protocol

Internet programming encompasses the technologies, protocols, and architectural patterns required to construct interactive, distributed software applications accessible via web browsers across the global Internet.
* Client-Server Paradigm: The web operates on a distributed request-response model. The Client (User-Agent / Web Browser) requests resources; the Server (e.g. Apache, Nginx, Node.js) processes business logic and serves documents.
* Uniform Resource Identifier (URI / URL): Syntax breakdown: scheme://host:port/path?query#fragment (e.g., https://lpu.in:443/courses?dept=cse#admissions).
* Domain Name System (DNS): Resolves human-readable domain names into machine-routable 32-bit IPv4 or 128-bit IPv6 IP addresses through a hierarchical lookup: Local Cache -> Recursive Resolver -> Root Server -> TLD Server -> Authoritative Name Server.
* Hypertext Transfer Protocol (HTTP/HTTPS): Stateless application-layer protocol operating over TCP/IP (port 80 for HTTP, port 443 for TLS-encrypted HTTPS).
- HTTP Request Methods: GET (retrieve data, idempotent, parameters in URL query string), POST (submit data in request body, non-idempotent), PUT (replace entire resource), PATCH (partial resource modification), DELETE (remove resource).
- HTTP Response Status Codes: 1xx (Informational), 2xx (Success: 200 OK, 201 Created), 3xx (Redirection: 301 Moved Permanently, 304 Not Modified), 4xx (Client Error: 400 Bad Request, 401 Unauthorized, 404 Not Found), 5xx (Server Error: 500 Internal Server Error, 503 Service Unavailable).

>> 1.2 The Document Object Model (DOM) and HTML5 Document Structure

* Anatomy of an HTML5 Document: <!DOCTYPE html> declaration triggers modern standards mode rendering. <html> root wraps <head> (metadata, title, character encoding UTF-8, responsive viewport) and <body> (renderable visual elements).
* The DOM Tree: The hierarchical tree structure generated by the browser rendering engine parsing HTML markup. Every tag becomes an Element Node; text inside tags becomes a Text Node; attributes become Attribute Nodes.

>> 1.3 Semantic HTML5 Elements vs. Generic Containers

HTML5 introduced semantic tags that describe their human and machine-readable meaning to search engines (SEO), screen-readers (accessibility), and developers, replacing ambiguous soup:
* Structural Semantic Tags: <header> (introductory content/logos), <nav> (navigation link blocks), <main> (dominant unique document content), <article> (self-contained syndicatable composition like blog posts), <section> (thematic grouping of content), <aside> (tangentially related content like sidebars), <footer> (copyright, author, contact data).
* Text Formatting and Phrasing Semantics: <h1> to <h6> (hierarchical headings), <p> (paragraphs), (strong importance/bold), (stress emphasis/italics), <blockquote>, <figure> and <figcaption>.

>> 1.4 HTML5 Advanced Form Controls and Client-Side Validation

Web forms (<form action='url' method='POST'>) capture structured user input:
* HTML5 Specialized Input Types: type='email', type='url', type='number', type='tel', type='date', type='time', type='range', type='color', type='file', type='password', type='checkbox', type='radio'.
* Built-in Constraint Validation Attributes: required (field cannot be empty), pattern='regex' (validates against regular expression), min and max (bounds for numeric/date inputs), minlength and maxlength, placeholder (ghost text guidance), autocomplete.
* Dropdowns and Grouping: <select> with <option> and <optgroup>, <datalist> (searchable autocomplete options for text inputs), <fieldset> and <legend>.

>> 1.5 HTML5 Native Multimedia Elements

* Video Embedding: <video width='640' controls poster='thumb.jpg'><source src='movie.mp4' type='video/mp4'></video>.
* Audio Embedding: <audio controls><source src='sound.mp3' type='audio/mpeg'></audio>.
* Canvas and SVG: <canvas> provides pixel-level procedural graphics via JavaScript 2D/WebGL context; <svg> provides resolution-independent vector XML graphics.

[PROGRAM 1: Accessible HTML5 Semantic Page Structure]

Problem Context: Construct a complete, standards-compliant HTML5 web document incorporating modern semantic structure, responsive viewport configuration, Open Graph

metadata, and accessibility tags.

HTML5 Source Code:

```
<!DOCTYPE html>
<html lang='en'>
<head>
<meta charset='UTF-8'>
<meta name='viewport' content='width=device-width, initial-scale=1.0'>
<meta name='description' content='LPU Department of Computer Science & Engineering'>
<title>School of Computer Science - Academic Portal</title>
</head>
<body>
<header>
<h1>Lovely Professional University</h1>
<nav aria-label='Main Navigation'>
<ul>
<li><a href='#about'>About Us</a></li>
<li><a href='#programs'>B.Tech Programs</a></li>
<li><a href='#contact'>Contact Admissions</a></li>
</ul>
</nav>
</header>
<main>
<article>
<header>
<h2>B.Tech in Computer Science and Engineering</h2>
<p>Published: <time datetime='2026-10-25'>October 25, 2026</time></p>
</header>
<section>
<h3>Curriculum Highlights</h3>
<p>The program integrates Full-Stack Web Development, Machine Learning, and Cloud
Systems.</p>
</section>
</article>
<aside>
<h3>Upcoming Hackathons</h3>
<p>Registration opens next Monday for the Annual Campus Coding Sprint.</p>
</aside>
</main>
<footer>
<p>&copy; 2026 Lovely Professional University. All rights reserved.</p>
</footer>
</body>
```

```
</html>
```

Key Elements: Includes DOCTYPE html, semantic layout (header, nav, main, article, section, aside, footer), and accessible time and ARIA attributes.

[PROGRAM 2: Advanced Interactive Student Registration Form]

Problem Context: Create a comprehensive HTML5 student registration form containing diverse input types, regex validation patterns, datalists, and custom file upload restrictions.

HTML5 Source Code:

```
<form action='/api/register' method='POST' enctype='multipart/form-data'>
<fieldset>
<legend>Student Personal Information</legend>

<label for='fullName'>Full Legal Name:</label>
<input type='text' id='fullName' name='fullName' required minlength='3'
placeholder='e.g. Aarav Sharma'>

<label for='regEmail'>Official University Email:</label>
<input type='email' id='regEmail' name='regEmail' required placeholder='name@lpu.in'>

<label for='regNo'>LPU Registration ID (8 Digits):</label>
<input type='text' id='regNo' name='regNo' required pattern='[0-9]{8}' title='Must be
exactly 8 numeric digits'>

<label for='semester'>Current Semester (1-8):</label>
<input type='number' id='semester' name='semester' min='1' max='8' value='1' required>

<label for='majorStream'>Engineering Stream:</label>
<input list='streams' id='majorStream' name='majorStream' required placeholder='Search
or select stream'>
<datalist id='streams'>
<option value='Computer Science and Engineering'>
<option value='Civil Engineering'>
<option value='Mechanical Engineering'>
<option value='Biotechnology Engineering'>
</datalist>

<label for='idProof'>Upload ID Document (PDF only, max 2MB):</label>
<input type='file' id='idProof' name='idProof' accept='.pdf' required>
```

```
</fieldset>
<button type='submit'>Complete Registration</button>
<button type='reset'>Clear Form</button>
</form>
```

Key Features: regex pattern='[0-9]{8}', datalist autocomplete, file accept restriction (.pdf), and semantic fieldset grouping.

[PROGRAM 3: Embedded Responsive Multimedia and Canvas API]

Problem Context: Build a web page integrating native HTML5 audio and video players with custom fallback messaging, alongside an HTML5 canvas drawing an interactive radar target.

HTML5 & JavaScript Code:

```
<!-- Native Multimedia Controls -->
<figure>
<video width='480' controls poster='preview.jpg'>
<source src='intro_cse.mp4' type='video/mp4'>
<source src='intro_cse.webm' type='video/webm'>
Your browser does not support the HTML5 video element.
</video>
<figcaption>Fig 1: Welcome Address by Dean of School of Engineering</figcaption>
</figure>

<!-- Interactive 2D Canvas Target -->
<canvas id='radarCanvas' width='200' height='200' style='border:1px solid
#333;'></canvas>
<script>
const canvas = document.getElementById('radarCanvas');
const ctx = canvas.getContext('2d');
// Draw concentric radar circles
ctx.strokeStyle = '#00aa00';
ctx.lineWidth = 2;
ctx.arc(100, 100, 40, 0, 2 * Math.PI); ctx.stroke();
ctx.beginPath(); ctx.arc(100, 100, 80, 0, 2 * Math.PI); ctx.stroke();
// Draw crosshair axes
ctx.beginPath();
ctx.moveTo(100, 10); ctx.lineTo(100, 190);
ctx.moveTo(10, 100); ctx.lineTo(190, 100); ctx.stroke();
</script>
```

Outcome: Renders cross-browser multimedia without Flash plugins and draws dynamic vector

circles via Canvas 2D API.

[PROGRAM 4: Structured Data Tables with Full Tabular Semantics]

Problem Context: Construct a semantically complete course gradebook table using caption, thead, tbody, tfoot, th with scope attributes, and column grouping.

HTML5 Source Code:

```
<table border='1'>
<caption>LPU CSE101 Semester Grade Roster - Autumn 2026</caption>
<thead>
<tr>
<th scope='col'>Reg No</th>
<th scope='col'>Student Name</th>
<th scope='col'>Mid-Term (30)</th>
<th scope='col'>End-Term (50)</th>
<th scope='col'>Total (100)</th>
</tr>
</thead>
<tbody>
<tr>
<td>12601145</td>
<td>Aarav Sharma</td>
<td>28.5</td>
<td>46.0</td>
<td>94.5</td>
</tr>
<tr>
<td>12601146</td>
<td>Diya Patel</td>
<td>24.0</td>
<td>42.5</td>
<td>86.5</td>
</tr>
</tbody>
<tfoot>
<tr>
<th scope='row' colspan='4'>Class Average Marks</th>
<td>90.5</td>
</tr>
</tfoot>
```

</table>
Key Features: Proper thead, tbody, tfoot separation, scope attributes for screen-readers, and colspan summary row.

>> 1.6 University Review & Examination Questions

* Q1: Differentiate comprehensively between GET and POST HTTP request methods in terms of data transmission, caching, idempotency, and security.
Answer: GET requests request data from a specified resource; form parameters are appended directly to the URL string as key-value pairs (query parameters). GET requests are idempotent (repeating identical requests causes no server state change), cached by browsers, bookmarked, and visible in browser history and server logs. Consequently, GET must NEVER transmit sensitive data (passwords, credit cards). POST requests submit data to be processed by a resource; data is transmitted within the HTTP request body. POST is non-idempotent (resubmitting creates duplicate records), never cached by default, not saved in history, and has no data length restrictions, making it mandatory for secure form submissions and database mutations.
* Q2: Explain the significance of Semantic HTML5 elements over non-semantic tags, highlighting impacts on Accessibility (A11y) and SEO.
Answer: Non-semantic tags like and tell the browser nothing about their contents, requiring arbitrary class names (e.g.) that differ between developers. Semantic elements (<header>, <nav>, <main>, <article>, <section>, <aside>, <footer>) carry explicit structural and functional meaning. For Accessibility, screen readers use semantic tags to provide landmark navigation shortcuts, allowing visually impaired users to jump directly to main content or skip navigation menus. For Search Engine Optimization (SEO), web crawlers parse semantic tags to determine information hierarchy and prioritize core article text over secondary sidebar advertisements.

>> 1.7 Key Takeaways and Revision Points

* HTTP is a stateless client-server request-response protocol running over TCP.
* Semantic HTML5 tags (<header>, <nav>, <main>, <footer>) improve accessibility and SEO.
* HTML5 form validation attributes (required, pattern, min/max) validate input without JS.
* The DOM represents the parsed document as an in-memory hierarchical object tree.